



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN
LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 04

NOMBRE COMPLETO: Cruz Cano Omar

N° de Cuenta: 318068932

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE: 2027-1

FECHA DE ENTREGA LÍMITE: 19 de septiembre de 2026

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa.

Terminar el rover: Cabina, base, dos llantas delanteras, brazo con pinza

Continuando con el código revisado en clase, implementé un segundo brazo articulado, así como un cono articulado en la punta que busca simular una herramienta perforadora, las articulaciones puede ser rotadas utilizando **F** para el primer brazo, **G** para el segundo y **H** para el cono. La implementación del segundo brazo resultó relativamente sencilla ya que solo fue necesario repetir código y ajustar un rotate, lo mismo para la articulación en la punta. Ajusté los colores a como yo consideré tiene un mejor aspecto (esferas y brazos de mismo color).

```
// Esfera
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 2.3f, 0.0f));
model = glm::rotate(model, mainWindow.getarticulacion3() * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
color = glm::vec3(1.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
sp.render();

// Pinza
model = modelaux;
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.0f, 1.3f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.5f, 2.0f, 0.5f));
color = glm::vec3(1.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[3]->RenderMeshGeometry();
```

Para la implementación de las 4 llantas traseras utilizamos una base en forma de cubo escalado en **X** para ser un prisma rectangular, luego un cilindro rotado 40 grados para poder movernos sobre **Y** como revisamos en clase, luego un segundo cilindro rotado 90 grados en **X** y escalado en **Y** para implementar el cilindro soporte, y por último otro cilindro con menor escala en **Y** y mayor escala en **X** y **Z** para la implementación de la rueda la cual tiene un color muy cercano a negro en todos los casos. Para implementar las otras 3 llantas solo fue necesario repetir código y modificar en los translates y rotates algún signo en donde fuera necesario, además cada soporte de llanta está pintado de un color diferente para lograr visualizarlos todos desde cualquier ángulo a ausencia de sombreado. Todas las llantas giran en la misma dirección utilizando la letra **J**.

```

//BASE PARA LA LLANTA 1
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -0.3f, 4.4f));
color = glm::vec3(0.5f, 0.5f, 0.5f);

modelaux2 = model;
model = glm::scale(model, glm::vec3(1.5f, 0.75f, 0.75f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[0]->RenderMesh();

// Cilindro llanta 1
model = modelaux2;
model = glm::translate(model, glm::vec3(-0.6f, 0.0f, 0.0f));
model = glm::rotate(model, -40 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(0.0f, -1.6f, 0.0f));
modelaux2 = model;
model = glm::scale(model, glm::vec3(0.35f, 3.5f, 0.35f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[2]->RenderMeshGeometry();

// Cilindro 2 llanta 1
model = modelaux2;
model = glm::translate(model, glm::vec3(0.0f, -1.45f, 0.4f));
model = glm::rotate(model, 40 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
modelaux2 = model;
model = glm::scale(model, glm::vec3(0.25f, 1.75f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[2]->RenderMeshGeometry();

```

Para las llantas delanteras, solamente se siguió la misma lógica que en las llantas traseras pero se tuvo que agregar dos cilindros mas para lograr la forma del soporte de las llantas delanteras solicitado. Todas las llantas giran con J.

```

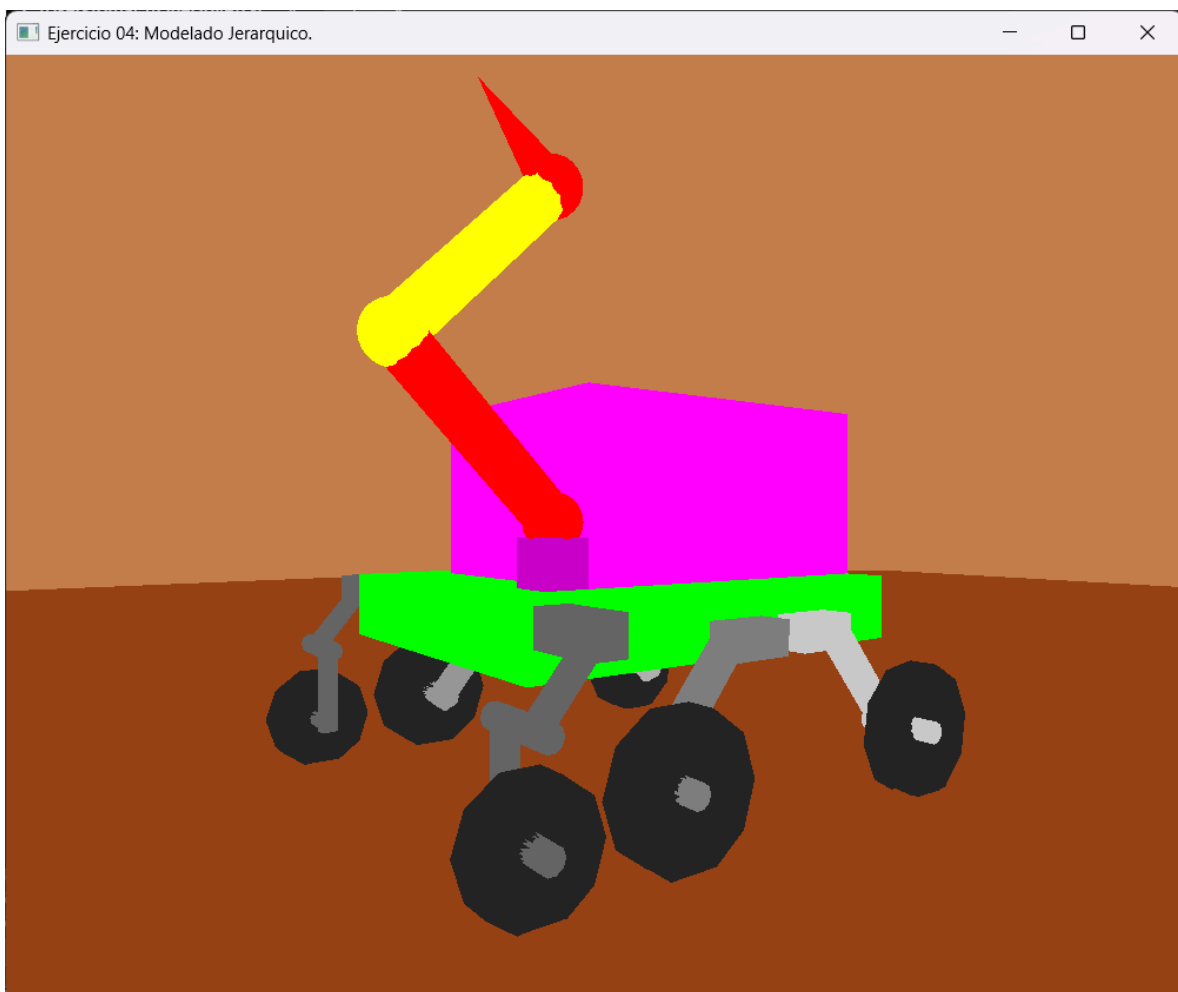
// Cilindro 2 llanta delantera 1
model = modelaux2;
model = glm::translate(model, glm::vec3(0.0f, -1.0f, -0.4f));
model = glm::rotate(model, 40 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
modelaux2 = model;
model = glm::scale(model, glm::vec3(0.25f, 1.75f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[2]->RenderMeshGeometry();

// Cilindro 3 llanta delantera 1
model = modelaux2;
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::translate(model, glm::vec3(0.0f, -1.0f, -0.5f));
modelaux2 = model;
model = glm::scale(model, glm::vec3(0.25f, 2.0f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[2]->RenderMeshGeometry();

```

Por último, para el piso repetimos el código utilizado en la práctica pasada solamente cambiando el color para simular cómo se vería el piso en Marte, además de que ajustamos el color del fondo para dar el efecto del cielo en Marte

```
// Piso marciano
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -1.05f, 0.0f));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
model = glm::scale(model, glm::vec3(150.0f, 0.55f, 150.0f));
color = glm::vec3(0.5882f, 0.2706f, 0.0784f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[0]->RenderMesh();
```



Problemas presentados. Listar problemas a la hora de ejecutar el código

Por no leer y no prestar atención, terminé implementando las 6 llantas completas.

Conclusión:

Esta práctica permitió consolidar los fundamentos del modelado jerárquico al estructurar un rover mediante relaciones padre-hijo, donde cada componente hereda y propaga de forma natural las transformaciones geométricas de su nodo predecesor. La principal complejidad técnica residió en coordinar adecuadamente el encadenamiento de matrices de modelo para los distintos grados de libertad como las articulaciones del brazo robótico (teclas **F**, **G** y **H**) y la rotación sincronizada de las ruedas (tecla **J**), asegurando que las traslaciones y rotaciones respetaran el punto de pivote correcto sin alterar la posición global del chasis; en general, el desarrollo de la sesión fue enriquecedor para afianzar la lógica de matrices acumulativas.